

Experiment No. 10

Aim:

Symmetric Encryption and Hashing.

Theory:

Symmetric Encryption:

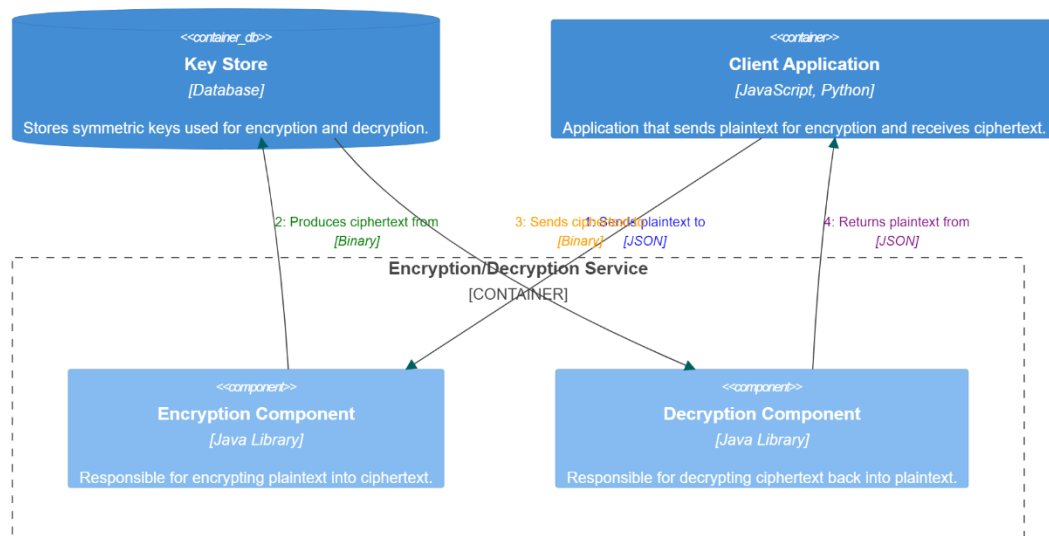
Symmetric Encryption is a type of encryption where the same key is used for both encryption and decryption. It ensures confidentiality by transforming readable data (plaintext) into an unreadable format (ciphertext) and requires the receiver to have the same secret key to convert the ciphertext back into readable data.

Key Components of Symmetric Encryption:

- Plaintext: The original, readable data that needs to be protected.
- Ciphertext: The encrypted, unreadable version of the plaintext.
- Encryption Algorithm: The mathematical process that transforms plaintext into ciphertext using a key. Common symmetric encryption algorithms include:
 - AES (Advanced Encryption Standard): A widely used symmetric encryption algorithm with key sizes of 128, 192, or 256 bits.
 - DES (Data Encryption Standard): An older encryption standard with a 56-bit key, now considered insecure.
 - 3DES (Triple DES): A stronger version of DES that applies the DES algorithm three times to each data block.

Key: A secret value known only to the sender and receiver, which is used to encrypt and decrypt the data.

Dynamic diagram for Symmetric Key Processing



1. **Encryption:** The sender uses the key to convert plaintext into ciphertext.
2. **Decryption:** The receiver uses the same key to transform the ciphertext back into plaintext.

Advantages of Symmetric Encryption:

1. **Fast:** Symmetric encryption algorithms are computationally efficient, making them suitable for encrypting large amounts of data.
2. **Simple Key Management:** Only one key is required for both encryption and decryption.

Disadvantages of Symmetric Encryption:

1. **Key Distribution Problem:** Both parties must securely exchange the key beforehand, which can be difficult in practice.
2. **Scalability:** As the number of participants increases, the number of keys needed grows exponentially. For n participants, $n(n-1)/2$ keys are required.

Hashing:

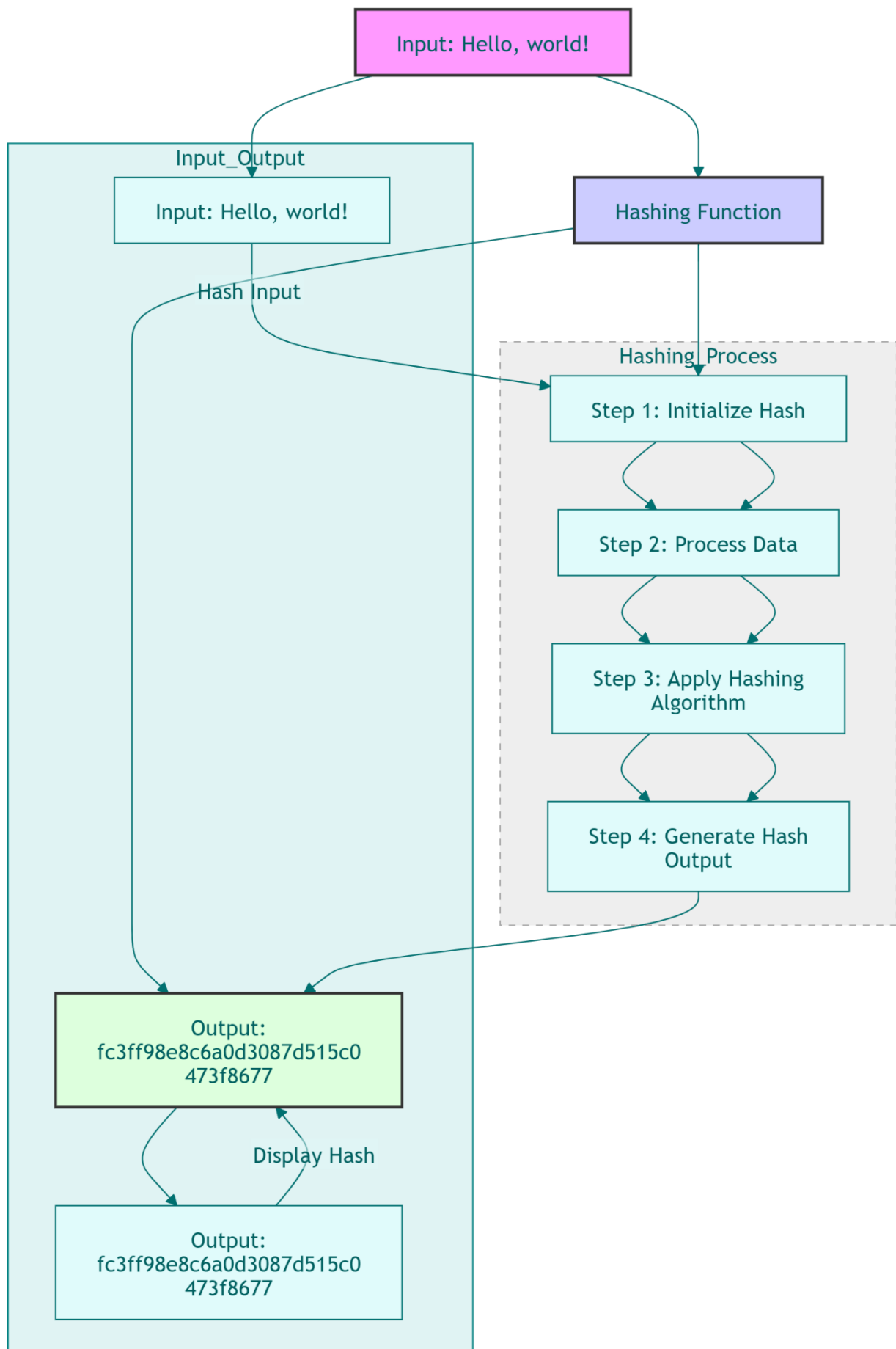
Hashing is a process that transforms any given input (usually data) into a fixed-size string of characters, which is typically a hash code. Unlike encryption, hashing is a one-way function—meaning that the data cannot be reverted to its original form.

Key Properties of Hashing:

1. **Deterministic:** The same input will always produce the same hash output.
2. **Fixed Size:** No matter the size of the input, the hash output will always have a fixed size (e.g., 256-bit output for SHA-256).
3. **Fast Computation:** Hash functions should be quick to compute.
4. **Pre-image Resistance:** It should be computationally infeasible to reverse the hash and retrieve the original input.
5. **Collision Resistance:** It should be computationally difficult to find two different inputs that produce the same hash value.
6. **Avalanche Effect:** A small change in the input should produce a significantly different hash.

Common Hashing Algorithms:

- **MD5 (Message Digest 5):** Produces a 128-bit hash. While fast, it is no longer considered secure due to vulnerability to collisions.
- **SHA-1 (Secure Hash Algorithm 1):** Produces a 160-bit hash. Also deprecated due to security vulnerabilities.
- **SHA-256 (Secure Hash Algorithm 256-bit):** Part of the SHA-2 family, produces a 256-bit hash and is widely considered secure.



Part 1: Symmetric Encryption Setup

1. Choose a file for encryption.

- Create or use a text file (**example.txt**) containing sensitive information.
- Example content: " **Aniket** "

2. Encrypt the file using a symmetric encryption algorithm (AES). Use the **openssl** command to encrypt the file using AES-256 encryption.

Bash : `openssl enc -aes-256-cbc -salt -in example.txt -out example.enc -k <password>`

Here, **<password>** is the encryption key (e.g., **mypassword123**).

3. Save the encrypted file (**example.enc**).

This file is now secured with symmetric encryption, and you cannot read the content without the correct key.

Part 2 : Simulating a Brute-Force Attack on Encryption by Hashing

4. Prepare for brute-force attack

Attackers usually start with a list of common passwords (such as **rockyou.txt**), attempting to decrypt the file using each password in the list.

5. Automate brute-force with a script

Create a brute-force script in Python or Bash to attempt decryption of **example.enc** using each password in the **rockyou.txt** list.

INPUT:

```
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.Scanner;

public class md {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the input message:");
        String input = sc.nextLine();
        String md5Hash = calculateMD5(input);
```

```

        System.out.println("Input: " + input);
        System.out.println("MD5 Hash: " + md5Hash);
    }

    public static String calculateMD5(String input) {
        try {
            MessageDigest md = MessageDigest.getInstance("MD5");
            md.update(input.getBytes());
            byte[] digest = md.digest();

            StringBuilder md5Hash = new StringBuilder();
            for (byte b : digest) {
                md5Hash.append(String.format("%02x", b & 0xff));
            }
            return md5Hash.toString();
        } catch (NoSuchAlgorithmException e) {
            e.printStackTrace();
            return null;
        }
    }
}

```

OUTPUT :

Output

```

java -cp /tmp/dddgW5Qzop md
Enter the input message:
Aniket
Input: Aniket
MD5 Hash: b86698d651a54a7d47919349daebd8be
|

```

Conclusion:

Thus, we have learnt and implemented Symmetric Encryption and Hashing using Brute Force Attack.